ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO THỰC HÀNH
VI XỬ LÝ – VI ĐIỀU KHIỂN

LAB 2: NGẮT TIMER VÀ QUÉT LED

(Timer Interrupt and LED Scanning)

Lớp: **L01** – HK261
Giảng viên hướng dẫn: **Lê Trọng Nhân**
Sinh viên thực hiện: **Đào Ngọc Thanh Tú – 2413841**

Link source code (GitHub)

`https://github.com/thanhtu988/bao-cao-vi-xu-ly-lab2`

Thành phố Hồ Chí Minh – Ngày 14 tháng 10 năm 2026

I. MÔI TRƯỜNG VÀ CẤU HÌNH

1. Công cụ sử dụng

Vi điều khiển	STM32F103C6Ux (Cortex-M3, HSI 8 MHz)
IDE / Sinh mã	STM32CubeIDE 1.7.0 + CubeMX 6.3.0, HAL FW_F1 V1.8.7
Mô phỏng	Proteus 8.10 SP0
Mã nguồn	10 project CubeIDE độc lập (`Lab2_Ex1` ... `Lab2_Ex10`)

2. Cấu hình TIM2 – chu kì ngắt 10 ms

Toàn bộ 10 bài dùng chung một cấu hình TIM2. Tần số ngắt tính theo công thức:

$$f_{\text{ngắt}} = \frac{f_{\text{clk}}}{(\text{PSC}+1)\times(\text{ARR}+1)} = \frac{8\,000\,000}{(7999+1)\times(9+1)} = \frac{8\times10^6}{80\,000} = 100\ \text{Hz}$$

$$\Rightarrow\ T_{\text{ngắt}} = \frac{1}{100} = 10\ \text{ms}$$

Tham số	**Giá trị**	**Ý nghĩa**
Clock Source	Internal Clock	lấy từ HSI 8 MHz qua APB1
Prescaler (PSC)	7999	chia 8000 → bộ đếm chạy 1 kHz
Counter Period (ARR)	9	đếm 10 nhịp 1 kHz → 10 ms
NVIC `TIM2_IRQn`	Enabled	bật ngắt toàn cục của TIM2

```
1  /* USER CODE BEGIN 2 */
2  HAL_TIM_Base_Start_IT(&htim2);
3  /* USER CODE END 2 */
```

Listing 1: Khoi dong ngat TIM2 trong main() – thieu dong nay thi callback khong bao gio chay

> **Lưu ý:** `HAL_TIM_Base_Start()` chỉ cho timer chạy, **không** bật ngắt. Phải dùng bản `_IT` thì CPU mới nhảy vào `HAL_TIM_PeriodElapsedCallback()`. Ngoài ra ARR phải ≥ 1: khi thử đặt ARR = 0 (chu kì 1 ms) thì Proteus mô phỏng CPU load lên hơn 50% và toàn bộ LED tối đen.

3. Quy ước mức logic

Tín hiệu	**Mức bật**	**Lý do phần cứng**
SEG0–SEG6 (PB0–PB6)	0	LED 7 đoạn **chung anode**
EN0–EN3 (PA6–PA9)	0	transistor **PNP** dẫn khi base ở mức thấp
DOT (PA4)	–	đảo trạng thái mỗi 1 giây
ENM0–ENM7 (cột matrix)	1	**ULN2803** là mảng transistor NPN
ROW0–ROW7 (PB8–PB15)	1	đường dữ liệu của LED matrix

```c
#define SEG_ON    GPIO_PIN_RESET   /* LED 7 doan chung anode */
#define SEG_OFF   GPIO_PIN_SET
#define EN_ON     GPIO_PIN_RESET   /* PNP dan khi base muc 0  */
#define EN_OFF    GPIO_PIN_SET
#define MATRIX_COL_ON   GPIO_PIN_SET   /* ULN2803 vao muc 1 */
#define MATRIX_ROW_ON   GPIO_PIN_SET
```

Listing 2: Macro muc logic dung chung cho ca 10 project

4. Phân bố chân

Ex	Chân sử dụng	SYS Debug
1	PB0–PB6 (SEG), PA6–PA7 (EN0–EN1), PA5 (LED đỏ)	Serial Wire
2–8	thêm PA8–PA9 (EN2–EN3), PA4 (DOT)	Serial Wire
9–10	thêm PA2, PA3, PA10–PA15 (ENM), PB8–PB15 (ROW)	**No Debug**

Lưu ý Ex9–Ex10: PA13/PA14 mặc định là SWDIO/SWCLK nên CubeMX không cho dùng làm GPIO. Phải đặt `SYS` → `Debug = No Debug`, khi đó CubeMX sinh `__HAL_AFIO_REMAP_SWJ_DISABLE()`. Hệ quả: nạp lại trên mạch thật phải dùng chế độ *connect under reset*.

5. Nguyên lý quét LED (LED scanning)

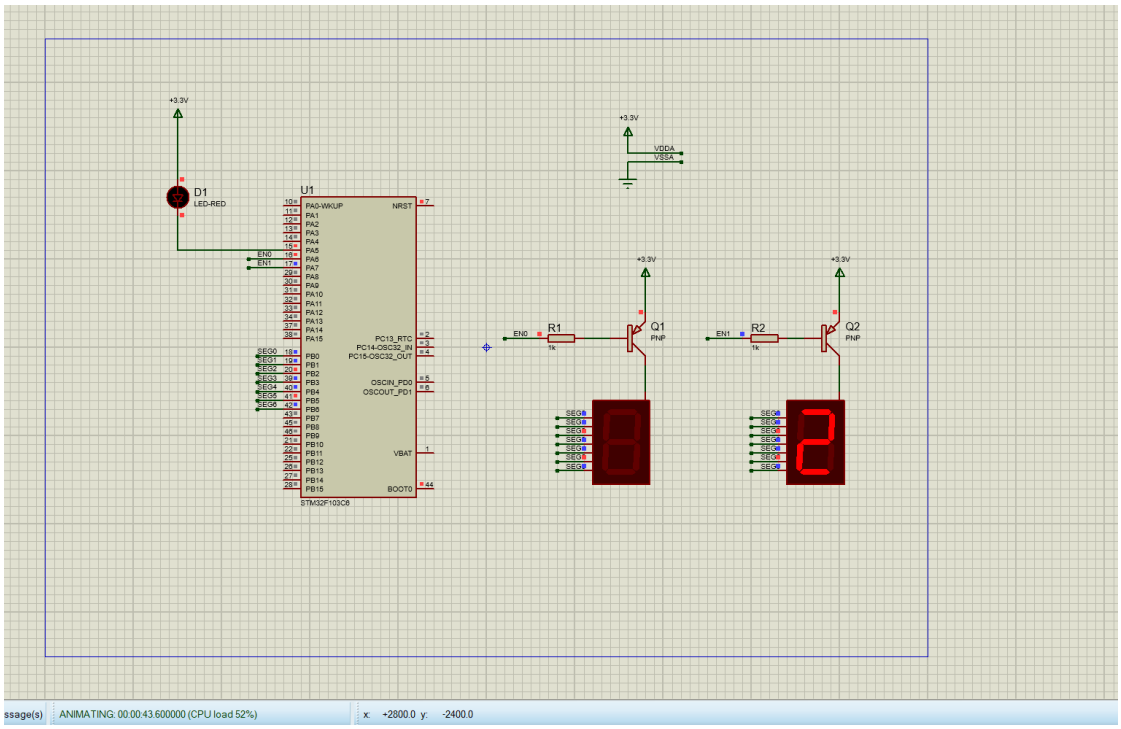
MCU chỉ có 7 chân dữ liệu nhưng phải điều khiển 4 LED 7 đoạn. Giải pháp: nối chung 7 đường SEG cho cả 4 LED, rồi dùng 4 chân EN bật **lần lượt từng LED một**. Mỗi lần quét luôn đi theo 3 bước:

Bước	Hành động	Nếu bỏ qua
1	Tắt **tất cả** EN	LED cũ còn sáng lúc đổi dữ liệu → bóng mờ (ghosting)
2	Xuất dữ liệu lên bus SEG	–
3	Bật **đúng một** EN	–

II. KẾT QUẢ 10 EXERCISE

Exercise 1 – Quét 2 LED 7 đoạn

Yêu cầu: LED thứ nhất hiện số 1, LED thứ hai hiện số 2, đổi qua lại mỗi 0,5 giây.



Hình 1: Exercise 1 – sơ đồ mô phỏng trong Proteus

Report 2 – mã nguồn trong hàm `HAL_TIM_PeriodElapsedCallback`:

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
  if (htim->Instance == TIM2) {
    timer_counter++;
    if (timer_counter >= TICKS_SWITCH) {      /* 50 x 10ms =
       500ms */
      timer_counter = 0;

      if (index_led == 0) {
        HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, EN_OFF);  /*
           B1: tat LED 2 */
        display7SEG(1);                                     /*
           B2: dat du lieu */
        HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, EN_ON);   /*
           B3: bat LED 1 */
      } else {
        HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, EN_OFF);
        display7SEG(2);
        HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, EN_ON);
```

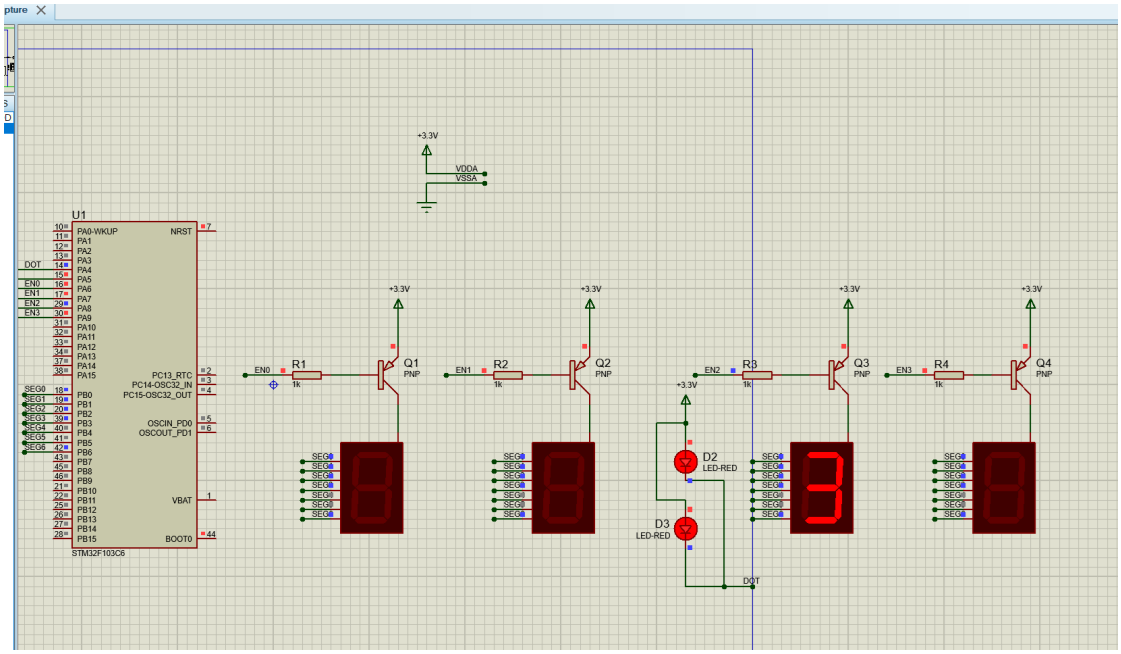
```
16        }
17        index_led = 1 - index_led;
18      }
19    }
20  }
```

Listing 3: Exercise 1 – callback ngat TIM2 (10 ms / lan)

Short question – tần số của quá trình quét?
Mỗi LED sáng 500 ms, có 2 LED nên một chu kì quét hết là $T = 2 \times 0{,}5 = 1$ s $\Rightarrow f = 1/T = \mathbf{1\ Hz}$.
Mắt người cần khoảng 24–50 Hz mới thấy liên tục, nên ở 1 Hz ta thấy rõ từng LED sáng lần lượt – đúng như mục đích minh hoạ của bài.

Exercise 2 – 4 LED 7 đoạn và 2 LED DOT

Yêu cầu: hiện 12:30 trên 4 LED, mỗi LED giữ 500 ms; 2 LED DOT nhấp nháy mỗi 1 giây.



Hình 2: Exercise 2 – sơ đồ mô phỏng trong Proteus

```c
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
  if (htim->Instance == TIM2) {

    /* --- Viec 1: quet 4 LED 7 doan, moi LED 500 ms --- */
    timer_counter++;
    if (timer_counter >= TICKS_SWITCH) {
      timer_counter = 0;
      HAL_GPIO_WritePin(EN0_GPIO_Port,
                        EN0_Pin | EN1_Pin | EN2_Pin | EN3_Pin,
                           EN_OFF);
      switch (index_led) {                          /* 12:30 */
        case 0: display7SEG(1);
                HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, EN_ON);
                   break;
        case 1: display7SEG(2);
                HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, EN_ON);
                   break;
        case 2: display7SEG(3);
                HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, EN_ON);
                   break;
        case 3: display7SEG(0);
                HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, EN_ON);
                   break;
```

```
20          default: break;
21        }
22        index_led++;
23        if (index_led >= 4) index_led = 0;
24      }
25
26      /* --- Viec 2: 2 LED DOT nhap nhay moi 1 giay --- */
27      dot_counter++;
28      if (dot_counter >= TICKS_DOT) {              /* 100 x 10ms =
           1s */
29        dot_counter = 0;
30        dot_state   = !dot_state;
31        HAL_GPIO_WritePin(DOT_GPIO_Port, DOT_Pin,
32                          dot_state ? GPIO_PIN_SET :
                             GPIO_PIN_RESET);
33      }
34    }
35  }
```

Listing 4: Exercise 2 – callback xu ly 2 viec doc lap: quet LED va nhap nhay DOT

Short question – tần số của quá trình quét?
$T = 4 \times 0{,}5 = 2$ s $\Rightarrow f = \mathbf{0{,}5\ Hz}$. Thêm LED mà giữ nguyên thời gian mỗi LED thì tần số quét **giảm đi** – đây chính là vấn đề mà Exercise 4 yêu cầu khắc phục.

Hai biến đếm độc lập: `timer_counter` và `dot_counter` đếm riêng nên hai công việc có chu kì khác nhau (500 ms và 1000 ms) vẫn chạy song song trên cùng một nguồn ngắt duy nhất.

Exercise 3 – Hàm `update7SEG(int index)`

Yêu cầu: gom phần quét thành một hàm, dữ liệu lấy từ mảng `led_buffer[4]`.

Report 1 – mã nguồn hàm `update7SEG`:

```
#define MAX_LED 4
int index_led = 0;
int led_buffer[MAX_LED] = {1, 2, 3, 0};     /* doi so o day la doi
   hien thi */

void update7SEG(int index)
{
  /* B1: TAT het 4 LED truoc. Neu bo buoc nay, du lieu cua LED cu
     con
         nam tren bus SEG trong luc LED moi da sang -> bong mo
            (ghosting). */
  HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin | EN1_Pin | EN2_Pin |
     EN3_Pin, EN_OFF);

  /* B2: dat du lieu  ->  B3: bat DUNG 1 LED */
  switch (index) {
    case 0:  display7SEG(led_buffer[0]);
             HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, EN_ON);
                break;
    case 1:  display7SEG(led_buffer[1]);
             HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, EN_ON);
                break;
    case 2:  display7SEG(led_buffer[2]);
             HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, EN_ON);
                break;
    case 3:  display7SEG(led_buffer[3]);
             HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, EN_ON);
                break;
    default: break;
  }
}
```

Listing 5: Exercise 3 – update7SEG() theo dung 3 buoc quet

Report 2 – mã nguồn trong `HAL_TIM_PeriodElapsedCallback`:

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
  if (htim->Instance == TIM2) {

    timer_counter++;
    if (timer_counter >= TICKS_SWITCH) {       /* 500 ms */
      timer_counter = 0;
      update7SEG(index_led++);
      if (index_led >= MAX_LED) index_led = 0; /* giu index trong
         0..3 */
```

```
    }

    dot_counter++;
    if (dot_counter >= TICKS_DOT) {            /* 1 s */
      dot_counter = 0;
      dot_state   = !dot_state;
      HAL_GPIO_WritePin(DOT_GPIO_Port, DOT_Pin,
                        dot_state ? GPIO_PIN_SET :
                           GPIO_PIN_RESET);
    }
  }
}
```

Listing 6: Exercise 3 – callback goi update7SEG(index_led++)

Hàm `update7SEG` tách phần *logic quét* khỏi phần *dữ liệu*: muốn hiện số gì chỉ cần ghi vào `led_buffer`, không phải sửa hàm. Đây là bước chuẩn bị cho Exercise 5 – đồng hồ chỉ việc ghi giờ/phút vào mảng này.

Exercise 4 – Nâng tần số quét lên 1 Hz

Yêu cầu: đổi chu kì gọi `update7SEG` để tần số của 4 LED 7 đoạn là 1 Hz. DOT vẫn nhấp nháy mỗi 1 giây.

$$f = 1\ \text{Hz} \Rightarrow T_{\text{vòng}} = 1\ \text{s} \Rightarrow T_{\text{mỗi LED}} = \frac{1000\ \text{ms}}{4} = 250\ \text{ms} \Rightarrow \frac{250}{10} = 25\ \text{nhịp ngắt}$$

```c
#define SCAN_FREQ_HZ      1
#define SWITCH_TIME_MS    (1000 / SCAN_FREQ_HZ / MAX_LED)   /* = 250 ms  */
#define TICKS_SWITCH      (SWITCH_TIME_MS / 10)             /* = 25 nhip */
#define DOT_TIME_MS       1000
#define TICKS_DOT         (DOT_TIME_MS / 10)                /* = 100 nhip */

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
  if (htim->Instance == TIM2) {

    timer_counter++;
    if (timer_counter >= TICKS_SWITCH) {    /* 25 x 10 ms = 250 ms */
      timer_counter = 0;
      update7SEG(index_led++);
      if (index_led >= MAX_LED) index_led = 0;
    }

    dot_counter++;
    if (dot_counter >= TICKS_DOT) {         /* 100 x 10 ms = 1 s */
      dot_counter = 0;
      dot_state   = !dot_state;
      HAL_GPIO_WritePin(DOT_GPIO_Port, DOT_Pin,
                        dot_state ? GPIO_PIN_SET :
                          GPIO_PIN_RESET);
    }
  }
}
```

Listing 7: Exercise 4 – chi doi mot hang so, logic giu nguyen

Viết hằng số dưới dạng biểu thức `1000 / SCAN_FREQ_HZ / MAX_LED` thay vì gõ thẳng số 25 giúp đổi số LED hay đổi tần số mục tiêu chỉ phải sửa một chỗ, và người đọc thấy ngay con số 25 từ đâu ra.

Exercise 5 – Đồng hồ số giờ:phút

Yêu cầu: hiện giờ và phút lên 4 LED 7 đoạn, theo khung chương trình thầy cho (dùng `HAL_Delay(1000)` trong `while`).

Report 1 – mã nguồn hàm `updateClockBuffer`:

```
int hour = 15, minute = 8, second = 50;
int led_buffer[MAX_LED] = {0, 0, 0, 0};

/* So 1 chu so tu dong co so 0 o truoc, vi du 8 phut -> "08" */
void updateClockBuffer(void)
{
  led_buffer[0] = hour   / 10;       /* hang chuc gio  */
  led_buffer[1] = hour   % 10;       /* hang don vi gio */
  led_buffer[2] = minute / 10;       /* hang chuc phut  */
  led_buffer[3] = minute % 10;       /* hang don vi phut */
}
```

Listing 8: Exercise 5 – updateClockBuffer() tach gio/phut thanh 4 chu so

```
  /* USER CODE BEGIN 2 */
  updateClockBuffer();               /* nap 15:08 len led_buffer
     truoc */
  HAL_TIM_Base_Start_IT(&htim2);
  /* USER CODE END 2 */

  while (1)
  {
    second++;
    if (second >= 60) { second = 0; minute++; }
    if (minute >= 60) { minute = 0; hour++;   }
    if (hour   >= 24) { hour   = 0;           }
    updateClockBuffer();

    HAL_Delay(1000);      /* delay CHAN - Exercise 6 se bo */
  }
```

Listing 9: Exercise 5 – vong lap chinh theo dung khung Program 1.5 cua thay

Phép chia và chia dư là cách tách chữ số kinh điển: $15/10 = 1$ lấy hàng chục, $15\%10 = 5$ lấy hàng đơn vị. Vì dùng chia nguyên nên $8/10 = 0$, số 0 đứng đầu được thêm *tự động* mà không cần câu lệnh `if` nào.

Phân vai rõ ràng: `main()` lo *nội dung* (giờ, phút), ngắt TIM2 lo *hiển thị* (quét 250 ms/LED). Hai bên chỉ trao đổi với nhau qua mảng `led_buffer`.

Exercise 6 – Software timer, bỏ HAL_Delay

Yêu cầu: tạo bộ đếm thời gian bằng phần mềm, đếm lùi mỗi lần ngắt, nhờ đó bỏ được `HAL_Delay(1000)` trong `main`.

```
#define TIMER_CYCLE   10          /* chu ky ngat TIM2, don vi ms */

int timer0_counter = 0;
int timer0_flag    = 0;

void setTimer0(int duration)
{
  timer0_counter = duration / TIMER_CYCLE;   /* doi ms -> so nhip ngat */
  timer0_flag    = 0;
}

void timer_run(void)
{
  if (timer0_counter > 0) {
    timer0_counter--;
    if (timer0_counter == 0) timer0_flag = 1;
  }
}
```

Listing 10: Exercise 6 – software timer theo dung Program 1.6

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
  if (htim->Instance == TIM2) {
    timer_run();
    /* ... phan quet LED va DOT van con o day ... */
  }
}

  /* USER CODE BEGIN 2 */
  setTimer0(1000);
  /* USER CODE END 2 */

  while (1)
  {
    if (timer0_flag == 1) {
      setTimer0(1000);          /* hen gio cho chu ky ke tiep */
      second++;
      if (second >= 60) { second = 0; minute++; }
      if (minute >= 60) { minute = 0; hour++;   }
      if (hour   >= 24) { hour   = 0;           }
      updateClockBuffer();
    }
```

```
23      /* Khong con HAL_Delay: trong luc cho, CPU hoan toan ranh */
24  }
```

Listing 11: Exercise 6 – ngat goi timer_run(), main() cho co timer0_flag

Report 1 – nếu thiếu dòng `setTimer0(1000)` ở đầu thì sao?

`timer0_counter` khởi tạo bằng 0. Hàm `timer_run()` có điều kiện `if (timer0_counter > 0)` nên nó **không bao giờ đếm**, `timer0_flag` mãi mãi bằng 0, thân lệnh `if` trong `while` không bao giờ chạy ⇒ đồng hồ (hoặc LED) đứng yên vĩnh viễn. Chương trình vẫn chạy nhưng không làm gì cả – lỗi *im lặng*, khó phát hiện vì không báo lỗi biên dịch. Lệnh `setTimer0` đầu tiên chính là cú *"lên cót"* cho vòng lặp.

Report 2 – nếu đổi thành `setTimer0(1)` thì sao?

`timer0_counter = 1 / TIMER_CYCLE = 1 / 10 = 0` vì đây là **phép chia nguyên** trong C. Kết quả giống hệt trường hợp Report 1: cờ không bao giờ lên, hệ thống treo. **Kết luận:** tham số truyền vào `setTimer0` phải ≥ `TIMER_CYCLE`; mọi giá trị trong khoảng 1–9 ms đều bị làm tròn xuống 0. Đây là giới hạn *độ phân giải* của software timer – không thể hẹn giờ ngắn hơn chu kì ngắt phần cứng.

Report 3 – nếu đổi thành `setTimer0(10)` thì khác gì hai trường hợp trên?

`10 / 10 = 1` nên counter = 1: đúng **một** lần ngắt sau (10 ms) là cờ lên 1 và vòng lặp chạy ngay. Khác hẳn hai trường hợp trên ở chỗ nó **có hoạt động**. Chỉ có chu kì đầu tiên bị rút ngắn còn 10 ms thay vì 1000 ms; các chu kì sau vẫn đúng vì bên trong `if` ta gọi lại `setTimer0(1000)`. Vậy 10 ms là **giá trị nhỏ nhất** mà software timer này còn phục vụ được.

Exercise 7 – Đưa DOT ra `main`

Yêu cầu: nâng cấp Exercise 5 bằng software timer, bỏ `HAL_Delay`, và chuyển luôn phần nhấp nháy DOT ra hàm `main`.

Report 1 – mã nguồn trong vòng lặp `while` của `main`:

```
  while (1)
  {
    if (timer0_flag == 1) {
      setTimer0(CLOCK_TICK_MS);          /* 1000 ms */

      second++;
      if (second >= 60) { second = 0; minute++; }
      if (minute >= 60) { minute = 0; hour++;   }
      if (hour   >= 24) { hour   = 0;           }
      updateClockBuffer();

      /* DOT da duoc chuyen tu ngat ra day */
      dot_state = !dot_state;
      HAL_GPIO_WritePin(DOT_GPIO_Port, DOT_Pin,
                        dot_state ? GPIO_PIN_SET :
                           GPIO_PIN_RESET);
    }
  }
```

Listing 12: Exercise 7 – dong ho va DOT cung nhip 1 giay nen dung chung 1 software timer

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
  if (htim->Instance == TIM2) {
    timer_run();

    timer_counter++;
    if (timer_counter >= TICKS_SWITCH) {
      timer_counter = 0;
      update7SEG(index_led++);
      if (index_led >= MAX_LED) index_led = 0;
    }
  }
}
```

Listing 13: Exercise 7 – callback nhe di mot nua: chi con timer_run() va quet LED

Đồng hồ và DOT đều có chu kì 1 giây nên chỉ cần **một** software timer; gộp chung vừa tiết kiệm biến, vừa bảo đảm DOT luôn đổi *đúng* lúc giây tăng – không bao giờ lệch pha.

Exercise 8 – Ngắt chỉ còn `timer_run()`

Yêu cầu: chuyển nốt `update7SEG()` ra `main`; ngắt timer chỉ còn nhiệm vụ phục vụ các software timer.

Report 1 – mã nguồn trong hàm `main` và các hàm phụ:

```
int timer0_counter = 0, timer0_flag = 0;
int timer1_counter = 0, timer1_flag = 0;

void setTimer0(int duration)
{ timer0_counter = duration / TIMER_CYCLE; timer0_flag = 0; }

void setTimer1(int duration)
{ timer1_counter = duration / TIMER_CYCLE; timer1_flag = 0; }

void timer_run(void)
{
  if (timer0_counter > 0) {
    timer0_counter--;
    if (timer0_counter == 0) timer0_flag = 1;
  }
  if (timer1_counter > 0) {
    timer1_counter--;
    if (timer1_counter == 0) timer1_flag = 1;
  }
}
```

Listing 14: Exercise 8 – them timer1, ham timer_run() dem ca hai bo dem

```
  /* USER CODE BEGIN 2 */
  updateClockBuffer();
  HAL_TIM_Base_Start_IT(&htim2);
  setTimer0(CLOCK_TICK_MS);        /* timer 0: dong ho + DOT
     (1000 ms) */
  setTimer1(SWITCH_TIME_MS);       /* timer 1: quet LED 7 doan (
     250 ms) */
  /* USER CODE END 2 */

  while (1)
  {
    /* ----- Timer 0: dong ho va DOT, moi 1 giay ----- */
    if (timer0_flag == 1) {
      setTimer0(CLOCK_TICK_MS);
      second++;
      if (second >= 60) { second = 0; minute++; }
      if (minute >= 60) { minute = 0; hour++;   }
      if (hour   >= 24) { hour   = 0;           }
      updateClockBuffer();

      dot_state = !dot_state;
```

```
      HAL_GPIO_WritePin(DOT_GPIO_Port, DOT_Pin,
                        dot_state ? GPIO_PIN_SET :
                           GPIO_PIN_RESET);
    }

    /* ----- Timer 1: quet LED 7 doan, moi 250 ms ----- */
    if (timer1_flag == 1) {
      setTimer1(SWITCH_TIME_MS);
      update7SEG(index_led++);
      if (index_led >= MAX_LED) index_led = 0;
    }
  }
```

Listing 15: Exercise 8 – main() dieu phoi 2 cong viec doc lap

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
  if (htim->Instance == TIM2) {
    timer_run();
  }
}
```

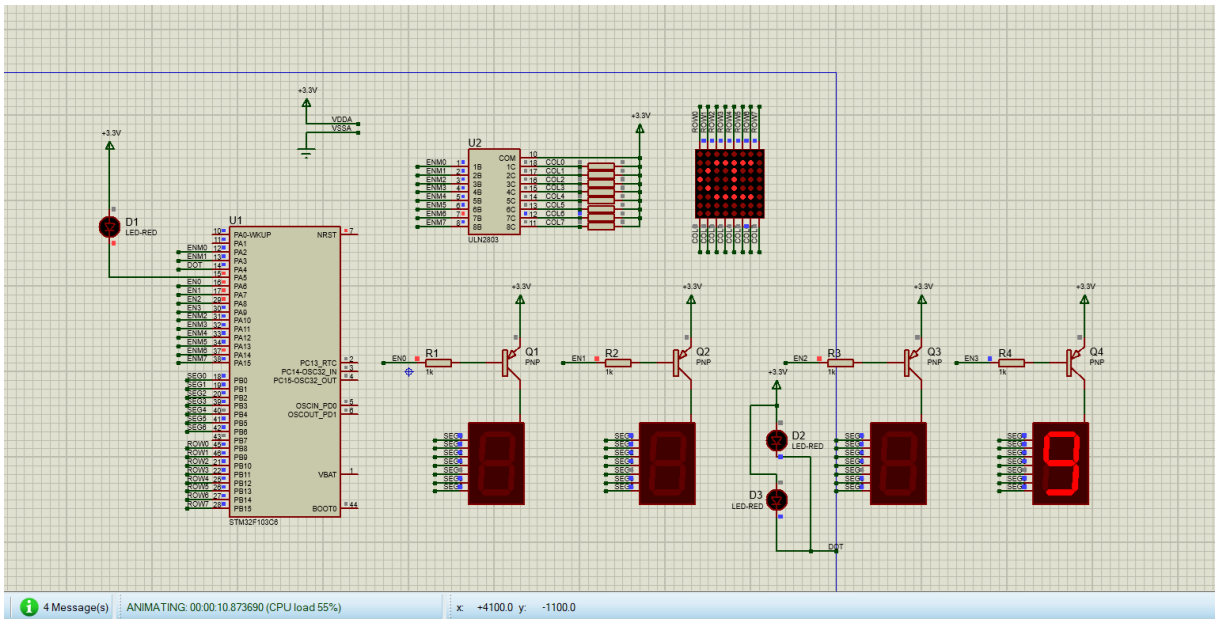
Listing 16: Exercise 8 – callback ngan nhat co the

Đây là đích đến của cả Lab 2. Ngắt chỉ còn vài chục chu kì lệnh nên không bao giờ chạy chồng lên nhịp sau, tránh được lỗi *nested interrupt*. Mọi tính toán nặng nằm ở `while(1)` – nơi có thể bị trì hoãn vài ms mà không gây hậu quả gì.

Mô hình này chính là một **cooperative scheduler** thu nhỏ: `timer_run()` đóng vai *tick* của hệ điều hành, mỗi khối `if (timerX_flag)` là một *task* có chu kì riêng. Thêm việc mới chỉ cần thêm một timer, không phải sửa dòng nào của các việc cũ.

Exercise 9 – LED matrix 8×8 hiện chữ "A"

Yêu cầu: thêm MATRIX-8X8-RED và ULN2803, cài hàm `updateLEDMatrix(int index)` và cho hiện chữ "A".



Hình 3: Exercise 9 – sơ đồ có LED matrix và ULN2803

Report 2 – mã nguồn hàm `updateLEDMatrix`:

```
#define MAX_LED_MATRIX 8
int index_led_matrix = 0;

/*         c0 c1 c2 c3 c4 c5 c6 c7
    ROW0     .  .  .  #  .  .  .  .
    ROW1     .  .  #  .  #  .  .  .
    ROW2     .  #  .  .  .  #  .  .
    ROW3     .  #  .  .  .  #  .  .
    ROW4     .  #  #  #  #  #  .  .       <- net ngang
    ROW5     .  #  .  .  .  #  .  .
    ROW6     .  #  .  .  .  #  .  .
    ROW7     .  .  .  .  .  .  .  .       net day 1 diem, chi chiem
       5 cot */
uint8_t matrix_buffer[MAX_LED_MATRIX] = {
  0x00, 0x7C, 0x12, 0x11, 0x12, 0x7C, 0x00, 0x00
};

void updateLEDMatrix(int index)
{
  uint8_t data = matrix_buffer[index];

  /* B1: tat TAT CA cot (neu khong se bi keo vet ngang) */
  HAL_GPIO_WritePin(ENM0_GPIO_Port,
```

```
                    ENM0_Pin | ENM1_Pin | ENM2_Pin | ENM3_Pin |
                    ENM4_Pin | ENM5_Pin | ENM6_Pin | ENM7_Pin,
                       MATRIX_COL_OFF);

  /* B2: dat du lieu 8 hang ROW0..ROW7 = PB8..PB15 */
  for (int r = 0; r < 8; r++) {
    HAL_GPIO_WritePin(ROW0_GPIO_Port, (uint16_t)(ROW0_Pin << r),
                      ((data >> r) & 1) ? MATRIX_ROW_ON :
                         MATRIX_ROW_OFF);
  }

  /* B3: bat DUNG 1 cot */
  switch (index) {
    case 0: HAL_GPIO_WritePin(ENM0_GPIO_Port, ENM0_Pin,
       MATRIX_COL_ON); break;
    case 1: HAL_GPIO_WritePin(ENM1_GPIO_Port, ENM1_Pin,
       MATRIX_COL_ON); break;
    case 2: HAL_GPIO_WritePin(ENM2_GPIO_Port, ENM2_Pin,
       MATRIX_COL_ON); break;
    case 3: HAL_GPIO_WritePin(ENM3_GPIO_Port, ENM3_Pin,
       MATRIX_COL_ON); break;
    case 4: HAL_GPIO_WritePin(ENM4_GPIO_Port, ENM4_Pin,
       MATRIX_COL_ON); break;
    case 5: HAL_GPIO_WritePin(ENM5_GPIO_Port, ENM5_Pin,
       MATRIX_COL_ON); break;
    case 6: HAL_GPIO_WritePin(ENM6_GPIO_Port, ENM6_Pin,
       MATRIX_COL_ON); break;
    case 7: HAL_GPIO_WritePin(ENM7_GPIO_Port, ENM7_Pin,
       MATRIX_COL_ON); break;
    default: break;
  }
}
```

Listing 17: Exercise 9 – ma chu "A": moi phan tu la MOT COT, bit i = ROW i

```
  setTimer0(CLOCK_TICK_MS);        /* 1000 ms - dong ho + DOT   */
  setTimer1(SWITCH_TIME_MS);       /*  250 ms - quet LED 7 doan */
  setTimer2(MATRIX_SCAN_MS);       /*   10 ms - quet LED matrix */

    /* ----- Timer 2: quet 8 cot LED matrix ----- */
    if (timer2_flag == 1) {
      setTimer2(MATRIX_SCAN_MS);
      updateLEDMatrix(index_led_matrix++);
      if (index_led_matrix >= MAX_LED_MATRIX) index_led_matrix =
         0;
    }
```

Listing 18: Exercise 9 – them timer2 quet matrix, goi tu main() nhu thay yeu cau

Chọn chu kì quét: 8 cột × 10 ms = 80 ms/khung ⇒ 12,5 khung/giây. Đây là giá trị nhỏ nhất mà software timer cho phép (bằng đúng `TIMER_CYCLE`), đủ để mắt thấy chữ liền. Nếu đặt `MATRIX_SCAN_MS = 100` thì một khung mất 800 ms – ta chỉ thấy từng cột sáng lần lượt chứ không ra chữ.

ULN2803 là transistor NPN nối kiểu *open-collector*: ngõ ra kéo xuống mát khi ngõ vào ở mức 1, còn khi mức 0 thì ngõ ra **thả nổi**. Vì vậy cột không được chọn sẽ tắt hẳn, và ta phải chủ động tắt tất cả các cột ở bước 1.

Exercise 10 – Hiệu ứng chạy chữ sang trái

Yêu cầu: tạo hiệu ứng động trên LED matrix, ví dụ chữ dịch sang trái.

Report 1 – mô tả giải pháp và mã nguồn:

> **Ý tưởng:** tách bạch *hiển thị* và *nội dung*. Hàm `updateLEDMatrix` không hề thay đổi so với Exercise 9 – nó luôn vẽ đúng những gì có trong `matrix_buffer`. Để chữ chạy, ta chỉ cần **xoay vòng** mảng `matrix_buffer` sang trái một cột sau mỗi 200 ms, bằng một software timer thứ tư. Cột đầu được đẩy vòng về làm cột cuối nên chữ chạy liên tục không bao giờ hết.

```
void shiftLEDMatrixLeft(void)
{
  uint8_t first = matrix_buffer[0];
  for (int i = 0; i < MAX_LED_MATRIX - 1; i++) {
    matrix_buffer[i] = matrix_buffer[i + 1];   /* cot i lay du
       lieu cot i+1 */
  }
  matrix_buffer[MAX_LED_MATRIX - 1] = first;   /* cot dau quay ve
     lam cot cuoi */
}
```

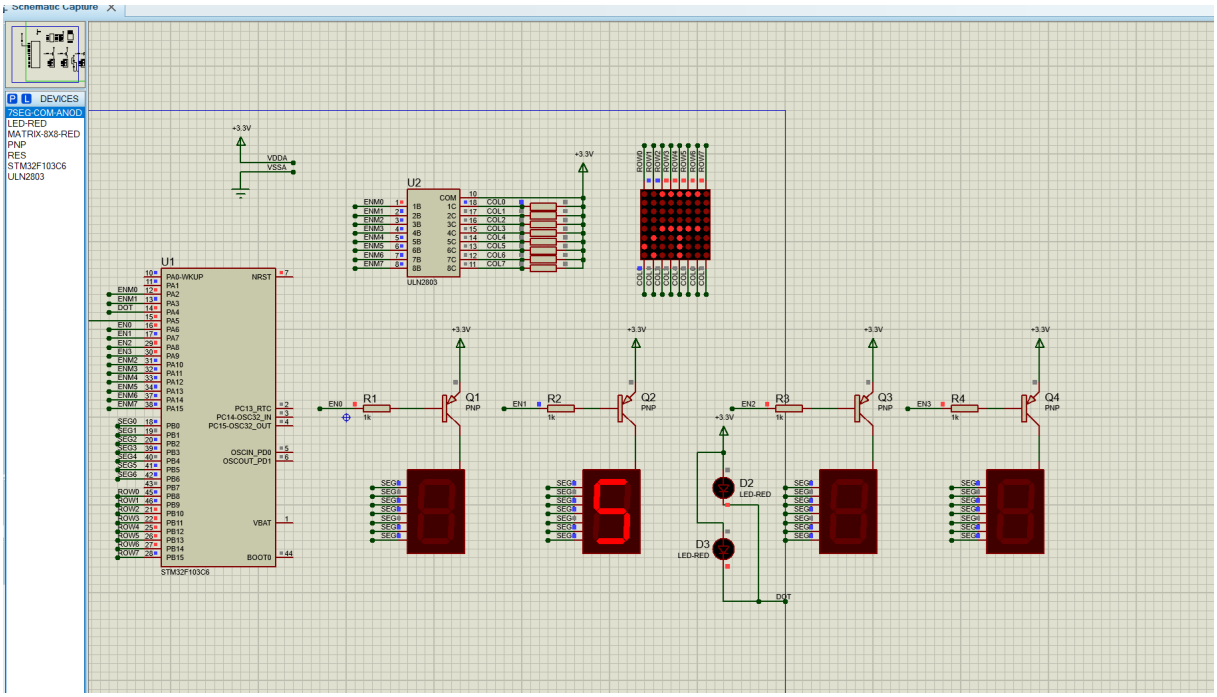
Listing 19: Exercise 10 – shiftLEDMatrixLeft() xoay vong mang 8 cot

```
#define SHIFT_TIME_MS   200        /* 200 ms / cot -> 5 cot moi
   giay */

  setTimer3(SHIFT_TIME_MS);

    /* ----- Timer 3: cu 200 ms dich chu sang trai 1 cot ----- */
    if (timer3_flag == 1) {
      setTimer3(SHIFT_TIME_MS);
      shiftLEDMatrixLeft();
    }
```

Listing 20: Exercise 10 – timer3 dieu khien toc do chay chu



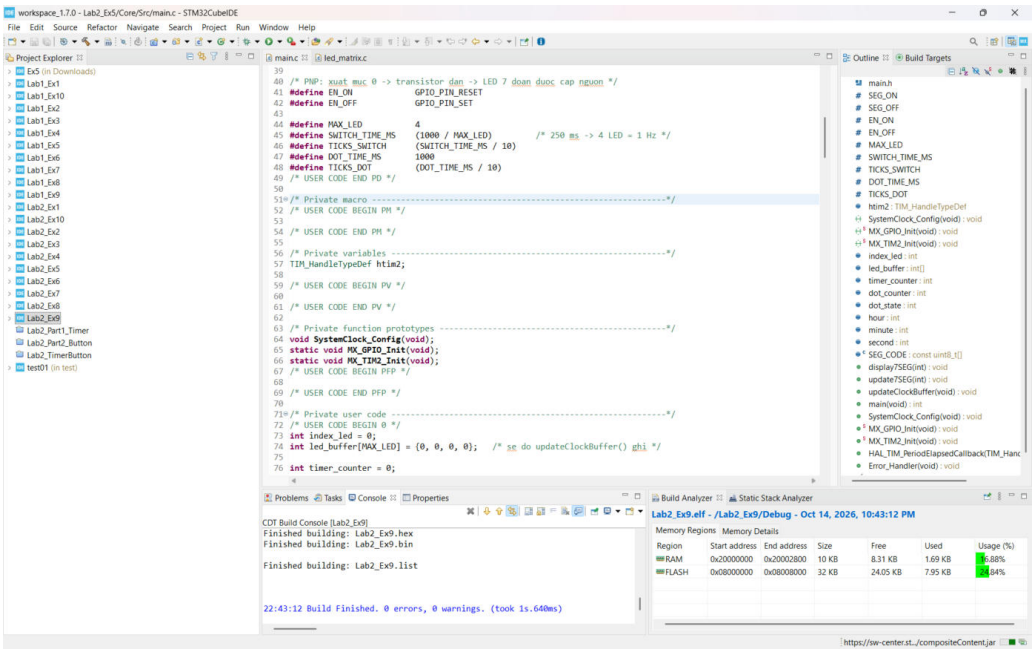
Hình 4: Exercise 10 – chữ "A" đang dịch sang trái

Tốc độ chạy chữ và tốc độ quét là **hai việc hoàn toàn độc lập**: muốn chữ chạy nhanh hơn chỉ cần đổi `SHIFT_TIME_MS`, không ảnh hưởng gì tới độ sáng hay độ mượt của màn hình. Đó là lợi ích trực tiếp của kiến trúc nhiều software timer ở Exercise 8.

III. TỔNG KẾT

1. Bảng kết quả

Ex	Nội dung	Chu kì quét	Build	Mô phỏng
1	Quét 2 LED 7 đoạn, hiện 1 và 2	500 ms	✓	✓
2	4 LED hiện 12:30, DOT nhấp nháy	500 ms	✓	✓
3	Hàm `update7SEG(int index)`	500 ms	✓	✓
4	Nâng tần số quét 4 LED lên 1 Hz	250 ms	✓	✓
5	Đồng hồ giờ:phút (`HAL_Delay`)	250 ms	✓	✓
6	Software timer, bỏ `HAL_Delay`	250 ms	✓	✓
7	Đưa DOT ra `main`	250 ms	✓	✓
8	Ngắt chỉ còn `timer_run()`	250 ms	✓	✓
9	LED matrix 8×8 hiện chữ "A"	10 ms	✓	✓
10	Chạy chữ sang trái	10 ms	✓	✓
Hoàn thành			**10/10**	



Hình 5: Build trong STM32CubeIDE – 0 errors, 0 warnings

2. Tư duy thiết kế rút ra sau Lab 2

Ex	Ngắt TIM2 làm gì	`main()` làm gì
1–4	quét LED + DOT + đếm thời gian	không làm gì
5	quét LED + DOT	đồng hồ (bị `HAL_Delay` chặn)
6	quét LED + DOT + `timer_run()`	đồng hồ (không bị chặn)
7	quét LED + `timer_run()`	đồng hồ + DOT
8–10	**chỉ** `timer_run()`	toàn bộ xử lý

- **Ngắt phải ngắn.** Thời gian nằm trong ISR là thời gian cả hệ thống bị "đóng băng". Lab 2 đi đúng một đường: dồn hết việc vào ngắt (Ex1) rồi gọt dần cho tới khi ngắt chỉ còn một dòng (Ex8).
- **Không dùng delay chặn.** `HAL_Delay` làm CPU đứng yên và cộng dồn sai số. Software timer thay nó bằng cơ chế "hẹn giờ rồi đi làm việc khác", là nền tảng của mọi RTOS.
- **Tách hiển thị khỏi dữ liệu.** `update7SEG` và `updateLEDMatrix` chỉ biết vẽ `led_buffer` và `matrix_buffer`; đồng hồ (Ex5) hay hiệu ứng chạy chữ (Ex10) chỉ việc ghi vào hai mảng đó. Nhờ vậy Ex10 thêm hiệu ứng mà **không sửa một dòng nào** của hàm quét.
- **Mỗi việc một chu kì riêng.** Một nguồn ngắt 10 ms duy nhất nuôi được bốn công việc có nhịp khác hẳn nhau (1000 / 250 / 10 / 200 ms) chỉ bằng bốn biến đếm lùi.
- **Liên hệ Lab 1:** Lab 1 đã viết `display7SEG()` và kết thúc bằng nhận xét `HAL_Delay` gây trôi thời gian. Lab 2 dùng lại nguyên hàm đó và trả lời đúng câu hỏi còn bỏ ngỏ: thay delay chặn bằng ngắt timer.

Source code:

`https://github.com/thanhtu988/bao-cao-vi-xu-ly-lab2`